# Chapter 6 - System Implementation

## Contents

## 6.1 Introduction

This chapter outlines the translation of the architectural and conceptual designs of the Intelligent Task Allocation System (ITAS) into a functional, robust software system. It details the development environment, the technologies utilized, and the step-by-step implementation of the core components. By examining the database schema, the backend logic, the NLP-based parsing engine, the algorithmic matching mechanisms, and the frontend user interface, this section demonstrates how the system's modules integrate to fulfill the core project objectives.

## 6.2 Development Environment and Setup

To ensure a stable and reproducible development environment, the project was developed using a modern technology stack across both the frontend and backend. The local development environment utilizes a dedicated DevDrive, a storage volume optimized for developer workloads, which reliably manages project files and effectively prevents pathing and compilation errors.

The backend server, NLP parsing engine, and matching algorithms were implemented using Python 3.12.10, leveraging its extensive data science and web development ecosystem. The web application frontend was built using React.js to provide a dynamic, component-based user interface. For persistent data storage, PostgreSQL was utilized to handle relational data securely and efficiently, operating seamlessly alongside Django's Object-Relational Mapping (ORM) capabilities.

## 6.3 Database Implementation

The database schema was implemented using Django's ORM, transforming Python class models into relational database tables (PostgreSQL in production, SQLite for local testing). The models are structured to capture complex relationships between Users, Employees, Skills, Projects, Tasks, and Task Assignments.

To maintain data integrity and prevent redundancy, strict constraints were implemented at the database level. For example, the Skill model maps an employee to their specific competencies. To prevent the system from assigning duplicate skills to a single employee (which could artificially inflate their suitability scores), a unique constraint is enforced:

python

```
class Skill(models.Model):
    """Skills extracted from CVs or manually added."""
    employee = models.ForeignKey(Employee, on_delete=models.CASCADE)
    name = models.CharField(max_length=100)
    source = models.CharField(
        max_length=20,
        choices=[('CV', 'CV'), ('MANUAL', 'Manual'), ('PORTFOLIO', 'Portfolio')],
        default='CV'
    )
    confidence_score = models.FloatField(default=0.0, help_text="Confidence score from NLP extraction")
    created_at = models.DateTimeField(auto_now_add=True)
    class Meta:
        unique_together = ['employee', 'name']
        ordering = ['-confidence_score', 'name']
```

The unique_together = ['employee', 'name'] directive in the Meta class ensures that an employee cannot have the exact same skill recorded twice in the database, inherently preventing data redundancy.

## 6.4 Backend and Core Algorithm Implementation

### 6.4.1 API Gateway and Routing

The backend acts as the central hub for data processing and serves as an API Gateway for the React frontend. It exposes RESTful endpoints built using the Django REST Framework, allowing the frontend to securely query data, upload documents, and trigger NLP processing. Routing is efficiently managed using a DefaultRouter which automatically maps standard CRUD operations.

python

```
from rest_framework.routers import DefaultRouter

from core.views import AuthView, EmployeeViewSet, TaskViewSet, ProjectViewSet

router = DefaultRouter()

router.register(r'employees', EmployeeViewSet, basename='employee')

router.register(r'tasks', TaskViewSet, basename='task')

router.register(r'projects', ProjectViewSet, basename='project')

urlpatterns = [

  path('auth/login', AuthView.as_view(), name='auth-login'),

  path('', include(router.urls)),

]
```

This concise routing implementation maps complex ViewSets to clear URL paths (e.g., /tasks/), establishing a standardized communication channel between the client and the server.

## 6.4.2 Document Parsing and NLP Engine

A critical requirement of ITAS is the automated extraction of information from employee CVs and task descriptions. The NLP Engine utilizes the PyPDF2 and spaCy libraries to parse text, extract entities, and identify roles. The text undergoes preprocessing before being passed to a pre-trained machine learning model to classify the employee's professional role.

python

```
@staticmethod
  def predict_role_with_ai(text: str, min_confidence: float = 0.45) -> Optional[str]:
    """Predict role using the trained AI model."""
    try:
      model = ModelLoader.get_model()
      if not model: return None
      cleaned = clean_text(text)

      if hasattr(model, "predict_proba"):
        probs = model.predict_proba([cleaned])[0]
        best_idx = int(probs.argmax())
        confidence = float(probs[best_idx])
        prediction = model.classes_[best_idx]
      else:
        prediction = model.predict([cleaned])[0]
      if confidence is not None and confidence < min_confidence:
        return None

      return prediction.replace("-", " ").title()
```

```
    except Exception as e:
        return None
```

This implementation loads a serialized machine learning model (.pkl), processes the raw CV text, and generates a probability array (predict_proba). If the confidence score meets the designated threshold, the algorithm classifies the employee's role, forming the foundation of their professional profile.

## 6.4.3 The Suitability Scoring Algorithm

The Matching Engine mathematically evaluates how well an employee aligns with a specific task. The algorithm computes a final suitability score (between 0 and 100) by considering multiple weighted factors: direct skill matching, skill coverage, skill experience (confidence level), current workload capacity, and historical performance ratings. The weights adapt dynamically based on the task's priority (e.g., skill match weighs heavier for High-priority tasks, while workload weighs heavier for Low-priority tasks).

python

```
def _calculate_total_score(
    self,
    employee: Employee,
    normalized_required: List[str],
    skill_profile: Dict[str, float],
    weights: Dict[str, float]
) -> float:
    performance_score = self._calculate_performance_score(employee)
    if not normalized_required:
        # Fallback if no specific skills are required
        workload_score = self._calculate_workload_score(employee)
```

```
        return (workload_score * weights["workload"] + performance_score *
weights["performance"])
    skill_score = self._calculate_skill_match_score(skill_profile, normalized_required)
    coverage_score = self._calculate_coverage_score(skill_profile, normalized_required)
    experience_score = self._calculate_experience_score(skill_profile,
normalized_required)
    workload_score = self._calculate_workload_score(employee)
    total_score = (
      skill_score * weights["skill"] +
      coverage_score * weights["coverage"] +
      experience_score * weights["experience"] +
      workload_score * weights["workload"] +
      performance_score * weights.get("performance", 0.0)
    )
    return max(0.0, min(1.0, total_score))
```

This segment highlights the formulaic aggregation of disparate metrics into a single, normalized confidence metric, ensuring objective and data-driven task allocation.

## 6.5 Frontend User Interface Implementation

The React.js frontend provides interactive dashboards for Project Managers and Employees. Using functional components and hooks (useState, useQuery), the UI handles complex states efficiently. When a Project Manager creates a task, the backend returns an array of matched employees, which the frontend renders dynamically as Recommendation Cards, presenting the data clearly for actionable decision-making.

tsx

```
<div className="divide-y divide-slate-100">
 {matches.map(match => (
  <div key={match.employee_id} className="p-4 flex items-center justify-between hover:bg-slate-50 transition-colors">
   <div className="flex items-center gap-4">
    <div className="w-10 h-10 rounded-full bg-indigo-100 text-indigo-700 font-bold flex items-center justify-center">
     {match.employee_name.charAt(0)}
    </div>
    <div>
     <div className="flex items-center gap-2">
      <h3 className="font-semibold text-slate-900">{match.employee_name}</h3>
      <span className="text-xs text-slate-400">{match.employee_title}</span>
     </div>
     <div className="flex items-center gap-2 mt-1">
      <div className="text-xs font-medium text-green-600 bg-green-50 px-2 py-0.5 rounded-full border border-green-100">
       {Math.round(match.suitability_score)}% Match
      </div>
      <div className="text-xs text-slate-400">
```

```
          {100 - Math.round(match.current_workload)}% Available
        </div>
       </div>
      </div>
    </div>
    <button onClick={() => handleAssign(match.employee_id)} className="btn btn-sm btn-outline">
      Assign
    </button>
   </div>
 ))}
</div>
```

This component effectively unpacks the JSON payload from the matching engine API, mapping over the matches array to visually display the employee's name, title, calculated suitability score, and current availability, alongside a direct action button to finalize the assignment.

## 6.6 System Integration

The system's modular components integrate securely to facilitate a continuous flow of data. A prime example is the Employee Onboarding Workflow:

User Interface (React): An employee accesses the web application and uploads their CV as a PDF document.

API Gateway (Django REST): The file is POSTed to the backend via a secure API endpoint.

NLP Engine (Python): The CVParser service utilizes PyPDF2 to read the file, and spaCy to extract entities and skills, while the machine learning model classifies their professional role.

Database (PostgreSQL): The extracted Skills, CV status, and updated Employee profiles are securely persisted into the relational database.

Algorithmic Engine: When a Project Manager generates a new task, the MatchingEngine queries the database, retrieves the newly processed employee skills, runs the suitability mathematics, and seamlessly returns the optimized recommendations back to the React UI.

## 6.7 Summary

The implementation phase successfully translated the theoretical design of the Intelligent Task Allocation System into a tangible product. By combining Python 3.12.10, Django, PostgreSQL, and React.js on a stable DevDrive environment, the system successfully integrates advanced Natural Language Processing and algorithmic matching within a responsive, user-friendly interface. The modular architecture ensures that data flows efficiently from initial ingestion through algorithmic analysis to actionable presentation, realizing the project's primary objectives.